

Lux AI S3 • Modeling and Solution Methods

From Problem Structure to Hungarian Allocation: the Trial-and-Error of Modeling (Part 2)

Bo Hou | jsjhoubo.github.io

Continuing the problem reading of Part 1, this part walks through modeling from problem structure to an implementable algorithm: accepting that most sub-problems have no closed-form optimum, and recording the wrong turns, concessions, and forced approximations along the way—the value of modeling often lies in knowing where there is no clean formula, and why.

1 Why Start With Reading the Problem

Lux AI Season 3 is an official NeurIPS 2024 competition: two teams each control a squad of units on a 24×24 grid, scoring by occupying hidden “point tiles”, best-of-five deciding the match. It is deliberately built around partial observability and meta-learning: each episode’s rule parameters and point-tile locations are unknown at the start, and a strong policy must infer them while playing.

Treated as an interview problem, the first step is not to write an agent but to **read the problem precisely**—translate every rule into mathematics, then see what kind of game this is and where it is hard. All mechanics are verified line-by-line against the official source (`env.py` / `state.py` / `params.py` in `luxai_s3`). A central conclusion: **“already this hard with an open hand”**—even if both players know where the point tiles are, optimal solving immediately falls into a PSPACE / NEXP / NP-hard encirclement, which is exactly why top solutions all go approximate (RL).

Bridge: the shift in viewpoint for Part 2. Part 1 ended by previewing a “keep pushing pure theory” route: the mixed-strategy equilibrium of Colonel Blotto, the information-theoretic lower bound of L0, a closed form $\sim H - d$ for tile value. The Blotto part has since been worked out separately (it is indeed a classic problem with a clean equilibrium). But moving the gaze back from “isolated sub-problems” to “how the whole agent should decide”, one thing becomes clear: **most of Lux’s sub-problems do not share Blotto’s luck**. Once local combat includes multiple steps, sap, and position, it degenerates into a simultaneous game, PSPACE or above, with no closed form; once the “unit→tile” assignment gains a temporal energy constraint on the path, it falls from a polynomial assignment problem into NP-hard.

In other words, *Blotto's solvability is the exception; unsolvability is the norm.*

So Part 2's viewpoint shifts from Part 1's “reading the problem + finding solvable structure” to “*modeling toward implementation, with complexity-awareness*”: accept that most sub-problems have no closed-form optimum, and instead honestly walk through “from problem structure to an implementable algorithm”, recording every wrong turn, every concession, every approximation forced by complexity along the way. That shift is itself what this article wants to convey—**the value of modeling often lies not in the final clean formula, but in knowing where there is no clean formula, and why.** So this part first lays out the problem structure and complexity boundaries (Sections 2–7, partly carried over from Part 1 for self-containment), then opens up the real trial-and-error trajectory (Section 9), and finally converges on Hungarian allocation as the implementable landing (Part 3 preview).

2 A Mathematical Statement of the Game Facts

Time and space. An episode has $K = 5$ matches of $H = 100$ steps; global time $\tau = (k - 1)H + t$. Map $\mathcal{C} = \{0, \dots, 23\}^2$, cells $c = (x, y)$. Two teams $p \in \{0, 1\}$, at most $N_{\max} = 16$ units each.

Units and actions. Unit state $u_{p,i}^{(\tau)} = (\text{alive}, \mathbf{pos}, e)$, energy $e \in [0, 400]$, init 100. Spawn corners are hard-coded at the diagonal $c_0 = (0, 0)$, $c_1 = (23, 23)$; one new unit every 3 steps, stacked on the corner. Action $a = (m, \delta x, \delta y)$: $m \in \{0, 1, 2, 3, 4\}$ moves one cell (stay/up/right/down/left), $m = 5$ is sap (a ranged energy strike). *Within a step all units' actions are submitted and resolved simultaneously; the two teams also act simultaneously.*

Hidden parameters. $\theta = (\mathcal{R}, \{\mathcal{G}_r\}, \rho)$, fixed across 5 matches, unknown at start: $\mathcal{R}$ relic positions ($|\mathcal{R}| \leq 6$, 1 $\sim$ 3 active + mirror); $\mathcal{G}_r$ a hidden 5×5 point-tile mask M_r per relic, each cell iid Bernoulli(≈ 0.2); ρ a vector of physical parameters (move_cost 1–5, sensor_range 1–4, sap_cost 30–50, sap_range 3–7, nebula vision/energy reduction, ...), drawn each episode—the physics of this particular episode.

Scoring (de-duplicated per cell). With $\text{occ}_p^{(\tau)}(c) = \mathbb{1}[\exists i : \text{an alive unit of } p \text{ on } c]$ and $\text{relic}(c) = \mathbb{1}[c \in \bigcup_r \mathcal{G}_r]$,

$$g_p^{(\tau)} = \sum_{c \in \mathcal{C}} \text{relic}(c) \cdot \text{occ}_p^{(\tau)}(c).$$

A tile scores +1 regardless of how many units stand on it; overlapping tiles count once. A cell covered by m relics has prior $P(\phi_c = 1) = 1 - 0.8^m$. A match goes to the higher $\sum_t g_p$

(ties: total energy, then random); the episode to more matches won.

Within-step resolution order (source-verified, crucial). submit $\rightarrow$ remove dead ($e \leq 0$) $\rightarrow$ spawn $\rightarrow$ move $\rightarrow$ **sap** $\rightarrow$ **collision** (same cell compares the two sides' *sum* of energy; lower side wiped, tie wipes both) $\rightarrow$ void $\rightarrow$ energy field $\rightarrow$ **scoring** $\rightarrow$ match check $\rightarrow$ emit per-team observations. Collision resolves *before* scoring: the winner clears the opponent off the cell and then scores, *but the kill itself grants no points*—this ordering matters greatly downstream.

Key information fact. In the observation `team_points` has shape 2 and is *unmasked*—**both totals are mutually public**; unit positions/energy are partially visible under the sensor mask, and the point-tile truth is fully hidden. The opponent's public score changes are thus an extra channel for inferring θ .

3 Information Structure: the Single Axis of Layering

Layering uses one axis—**what both players know about the point tiles ϕ** . That totals are public, units partially visible, opponent information hidden, are environment-given constants, not axes. The referee is omniscient, does not compete, does not enter the layering.

Layer	Both players' knowledge of ϕ	Game type
L1	known	complete-info, simultaneous two-player Markov game
L2	unknown (inferred from totals)	incomplete-info two-player game (real Lux)

L0 is not a layer. “Inferring ϕ from totals” is a *tool* inside L2: cells ϕ_c are independent a priori, but $y = \sum_{c \in S} \phi_c$ is a sum, the likelihood does not factorize, so the posterior *couples* jointly-observed cells—group testing / compressed sensing. We once treated L0 as a sibling layer and the “L0–L1 boundary” stayed muddy; the fix: *L0 is a tool, not a layer*, and layering uses only the single axis “known-or-not”; mixing in “explored-or-not” as a second axis is what blurs it.

The global optimum targets L2, but the solution path is $L0 \rightarrow L1 \rightarrow L2$. Rule-based does explicit assembly; RL approximates the same L2 optimum end-to-end—aligned because they share the *same information bottleneck* (only totals visible).

4 A Modeling Choice That Decides Difficulty: Centralization

A team has many units—one brain controlling many, or many independent agents? An implementation choice, and it fixes the complexity class:

Modeling	Decision-makers/team	Overall model
Centralized (one brain)	1 (large joint action space)	two-player POSG / POMDP vs fixed opp.
Decentralized (independent)	N	Dec-POSG

Lux’s act takes the *whole team’s* observation and outputs the *whole team’s* action—so full centralization (both training and execution, matching Kaggle’s single-agent interface) is allowed. It drops the problem from Dec-POMDP (NEXP-complete) to a two-player POSG ($\sim$ PSPACE) and keeps L1→L2 a *two-player* game throughout. The cost is a joint action space $\leq 6^{16}$ —a *computational* issue (decomposition/heuristics), not the *structural* hardness of Dec-POMDP. Centralization trades “coordination is hard” for “the action space is large”. All layers here assume centralization; decentralization (CTDE: centralized training, decentralized execution—QMIX / MADDPG) is left for later comparison.

5 Complexity Placement: a Wall of Barriers Under an Open Hand

This section collects the complexity conclusions. Note: except for a few comparison rows, **most of the hardness appears already in L1 (open hand)**, not from hidden tiles.

The ladder (proven inclusions). $P \subseteq NP \subseteq PSPACE \subseteq EXPTIME \subseteq NEXP$. Only the two ends $P \subsetneq EXPTIME$ and $NP \subsetneq NEXP$ are proven strict; every inner cut (P vs NP, NP vs PSPACE, PSPACE vs EXPTIME) remains *open*. From the strict ends we know “at least one cut in the chain is strict, but we cannot point to which”—a core research direction of complexity theory.

Object	Complexity	Note
Centralized Lux (2-player POSG)	$\sim$ PSPACE	game's simultaneous/alternating structure, $\exists$
Decentralized Lux (Dec-POMDP)	NEXP-complete	many decision-makers + no comms
Go/Chess ($n \times n$, ref.)	EXPTIME-complete	complete info but tree blowup, provably exp
Assignment + path energy-timing	general ILP, NP-hard	TU structure destroyed
Pure assignment / max-flow / bipartite	polynomial	totally unimodular, LP relaxation is integral

Why assignment is a “polynomial island”. A 0/1 assignment is writable as an ILP (NP-hard in general), but its constraint matrix is *totally unimodular*—the LP relaxation is automatically integral, so one solves the LP directly, polynomially (Hungarian $O(n^3)$). Add a *temporal* hard constraint like “energy stays positive at every step of the path” and TU is broken, collapsing to a general ILP, NP-hard. This dividing line recurs in Part 2's cost modeling: *soft constraints* (*unreachable* $\rightarrow \infty$) *keep it polynomial; hard temporal budgets fall into NP*.

Why approximation ratios do not apply. NP-hard optimization often admits “within $X \times$ optimum” guarantees. But here “optimum” is a *game's Nash value*, not a single-objective optimum; a non-zero-sum game has no single “optimal value” to compare against. So for local combat we chase *no ratio*, only conservative sufficient conditions (bounds). Ratios are a tool for combinatorial optimization and do not transfer to games.

Game DP blowup and “freezing the opponent”. With both sides simultaneously optimal, each DP state must solve a simultaneous matrix game, and the state space blows up with units/energy/positions—*infeasible*. A common approximation *freezes the opponent* (assume no move this step), reducing combat to single-agent optimization and a single-agent DP. But: *freezing one side is exactly the “alternation” of a game tree* (chess freezes the opponent every move); and Lux is *simultaneous*, both sides acting on the other's old state, which *violates the alternating game-tree assumption*. So in Lux freezing approximates a simultaneous game by an alternating one—optimistic, not exact. The real workhorse across “simultaneous games are unsolvable” is **self-play** (alternating best responses iterating toward equilibrium)—the AlphaGo/AlphaZero idea, deferred to Part 2's RL chapter.

6 The Game-Theoretic Framework for L1

A game $\Gamma = \langle \mathcal{N}, \{A_i\}, \{u_i\} \rangle$. Nash equilibrium: (a_1^*, a_2^*) where neither can improve *their own* utility by deviating,

$$u_1(a_1^*, a_2^*) \geq u_1(a_1, a_2^*) \forall a_1, \quad u_2(a_1^*, a_2^*) \geq u_2(a_1^*, a_2) \forall a_2.$$

Both conditions point the *same direction* (each maximizes its own u_i)—keeping this form avoids flipping an inequality in the zero-sum single-function max / min translation (a pitfall actually hit and fixed).

Checking vs inferring. *Checking* an equilibrium needs only the definition. The “misuse” happens in *inference*: applying properties (unique value, equals minimax, predicts the winner, is optimal) without verifying the precondition (zero-sum?). Discipline: for any equilibrium property, return to the precondition and prove it.

Nash value in plain words. In a zero-sum game interests are fully opposed, and under mutual optimality the outcome is *uniquely determined*—the number I can guarantee is the Nash value $V = \max \min = \min \max$ (the two sides squeeze the same value), whose sign decides the winner. Not a “minimum point” but a *value* at a saddle.

L1’s true structure: non-zero-sum scoring + zero-sum verdict. The scoring process is *non-zero-sum*—both occupy non-conflicting cells, $G_A + G_B$ not constant, A gaining one does not cost B one; only contesting the *same* cell is locally zero-sum ($V = \text{sign}(e_1 - e_2)$). But the verdict $\mathbb{K}[G_A > G_B]$ is zero-sum. Each side’s objective is a composite: maximize own occupation efficiency (non-zero-sum) + suppress the opponent in contests (zero-sum). Since only the sign matters, *excess points are worthless*—defend ahead, gamble behind. Precisely because the non-zero-sum part has no single value, approximation ratios are moot.

7 A Derived Atom: Single-Cell Contest (one-step 2×2)

Minimal combat unit: one point tile g , each side one unit already adjacent, simultaneously deciding step-in (I) or not (N).

Utility holds only immediate score. Each step’s reward comes only from occupation; kills, energy changes, unit losses *do not enter immediate reward*—they belong to the state layer / future value. Collision resolves before scoring but merely clears the cell for the winner; the kill adds nothing. We once tried adding a “unit value w ” to the single-step matrix and

got *two NE with different values*—but all NE of a zero-sum game must share one value; the contradiction reverse-proves w belongs to the multi-step future value, not the single-step matrix. With u = player 1’s score difference:

Energy	$u(I, I)$	$u(I, N)$	$u(N, I)$	$u(N, N)$	pure NE (value)
P1 higher	+1	+1	−1	0	$(I, I), (I, N)$ (+1)
Equal	0	+1	−1	0	(I, I) (0)
P1 lower	−1	+1	−1	0	$(I, I), (N, I)$ (−1)

When P1 is higher, I strictly dominates; when equal, (I, I) is the unique equilibrium (not stepping in lets the opponent take it free, so one steps in even into mutual destruction—a deterrence equilibrium); when lower, contesting loses. The two equilibria share one value, consistent with “zero-sum NE value is unique”. **Principle:** contest only if $e_{\text{self}} \geq e_{\text{opp}}$; at a disadvantage do not contest—conserve the unit for uncontested cells. This is the first rule derived from game theory that goes straight into the agent.

8 From Open Hand to Reality: Two Task Types

L1 (open hand) is only the foundation and is not submittable: real tiles are hidden. A real agent’s units split into *occupy* (hold a discovered tile) and *explore* (lift fog, find tiles). Both compete for the same finite units—the concrete form of exploration-exploitation: of these 16 units, how many occupy, how many scout. A single Hungarian can handle it: a task pool mixing tiles and exploration points, each with its own reward (occupy = tile value, explore = information gain). L1 is the special case “only occupy”; Part 2 first gets occupy right, deferring the information-gain reward to the L0 chapter.

9 The Trial-and-Error Trajectory of Modeling

This section lays bare the “went-wrong / got-stuck / fixed-it” that actually happened during the derivation. For an article about modeling, these back-and-forths are worth more than the final conclusions: they reveal which intuitions mislead, which paths are dead ends, and why one stops where one does.

1. The disillusionment of seeking a closed form. The first instinct was to find a “globally optimal policy”—given any state, compute how both sides should play under mutual

optimality. This works for things like the simplex method or max-flow (clean closed-form or polynomial solutions), so it felt natural to try it on Lux. But Lux’s global optimum lives in L2, a two-player, partially observable, simultaneous-move stochastic game (POSG), whose exact Nash equilibrium is PSPACE-level or higher—*no attainable closed form exists*. Recognizing this is the methodological turning point: stop chasing a “global closed-form optimum”, pursue instead “locally solvable sub-games + global approximation”. The single-cell contest (2×2) has a closed form precisely because it is pared to the minimum; scale up to the full game and the closed form vanishes.

2. Action space: free allocation vs graph constraint. For a while “assign a unit to any point tile” was taken as the action—but a unit moves one cell per step and cannot teleport to a distant tile. So the real action is a *position-constrained* local move, with an edge between a unit and a tile only if it is reachable (in one or several steps). This turns allocation from “free assignment” into *assignment under a graph constraint*, and incidentally refutes the analogy to classic Colonel Blotto (free troop allocation)—Lux’s units are locked by position, a graph-constrained variant of Blotto.

3. The cost dimension, flip-flopping. What should the cost be? First “Manhattan distance”, then “net energy”, then briefly “distance – energy”—the last rejected: distance and energy consumption are positively correlated (moving d steps costs $d \cdot \text{move_cost}$), two expressions of the same thing, and subtracting them is meaningless. Finally fixed as *steps for cost, energy as a feasibility gate*: optimize “arrive fastest”, with energy only judging “enough to reach” (else ∞). This “one as objective, one as gate” separation is the baseline for all later cost modeling.

4. Single vs two layers, and “recompute each step”. If each step runs Hungarian only over “adjacent cells”, units are myopic (they see only the cell underfoot, not which distant tile to head for). So add a layer: an outer layer (every T steps) assigns unit→tile target, an inner layer (each step) moves one cell under that target. And because the opponent changes every step and old assignments go stale, we adopt *recompute each step* (outer on a fixed period, inner every step) rather than “assign once and walk to the end”.

5. Focus fire: many-to-one breaks Hungarian’s one-to-one. Standard assignment is one-to-one (one unit, one target). But “several units focus-fire one tile / one enemy” is *many-to-one*, which Hungarian does not directly support. Exact handling expands to min-cost flow or multiple slots per tile; the first version’s trade-off is to *keep one-to-one and handle focus fire with a separate greedy rule*, not swapping in a heavier model for its sake.

6. The boundary of DP: single-agent feasible, game explodes. We considered DP for local combat. The conclusion splits in two: if the opponent is *frozen* (treated as a fixed environment), “how I move and when I sap” is a single-agent multi-step decision and *single-agent DP is feasible*; but if both sides are optimal, every state must solve a simultaneous matrix game and *game DP explodes, infeasible*. A deeper realization: freezing one side is exactly the “alternation” of a game tree (chess freezes the opponent every move), whereas Lux is *simultaneous*, both sides acting on the other’s old state, which *violates the alternating game-tree assumption*. So in Lux “freezing” approximates a simultaneous game by an alternating one—optimistic, computable, but not exact.

7. Approximation ratio: to chase or not? Given the NP-hard flavor, one naturally asks “how many times optimal is this approximation”. But approximation ratios are designed for *single-objective optimization* (with a well-defined optimum to compare); here the “optimum” is a *game’s Nash value*, and the non-zero-sum part has no single optimal value at all. So the ratio framework *does not fit* and is not chased—not laziness, but a tool-problem mismatch. Local combat gets only conservative sufficient conditions (bounds).

8. The sap timing correction. It was briefly taken for granted that “this step sap the opponent away, and the same step another unit walks in to occupy”. But the source’s resolution order is move→sap→collision→score, *simultaneous*, not a two-beat “clear then occupy”: the unit walking in this step faces the energies after both sides have moved this step. So “sap + occupy” within a step is coupled—either rely on the collision energy sum, or split across steps (weaken this step, occupy next); there is no “single-step clear-then-occupy”.

9. The expedient of bypassing value. Sap itself scores nothing; its payoff is in the future (occupying after clearing). To compute sap’s payoff exactly, one must first define each tile’s value $value(g)$ —which pulls in value-function modeling, exactly what RL learns. The rule version’s trade-off: *bypass value with greedy thresholds* (“if collision is short on the target tile, focus-fire nearby; if a held tile is attacked, sap the attacker away; surplus units sap around the tile”), leaving precise value estimation to the RL chapter.

10. Own-unit path conflicts. Combat kept computing “collision with the enemy” and nearly missed that *one’s own units* can block each other on the same path. The handling is heuristic: many tiles → units disperse, rarely colliding; few tiles → greedy nearest; still blocked → let “nearest-to-target move first” break it. No chasing MAPF / CTAPF optimality—that is yet another hard problem.

10 Part 2 Preview

Part 2 lands the **rule-based Hungarian allocation**, theory and experiment together, split into 2.1 / 2.2 / ... as needed:

- **Per-step inner/outer layering:** the outer layer (every T steps) assigns unit→tile target, cost = steps, energy as a feasibility gate (unreachable $\rightarrow \infty$, soft constraint keeps it polynomial); the inner layer (every step) picks “adjacent cell or stay + whether to sap” under the target.
- **Cost dimension:** after reversals (distance? net energy? “distance – energy” rejected for mixed dimensions), fixed as *steps for cost, energy for the gate*; arrival energy is path-dependent, taken as the upper bound of the energy-optimal path via BFS/shortest path (optimistic).
- **Combat feasibility (no closed form, use bounds):** for a defended tile, compare “our arrival energy (minus opponent sap)” with “defender energy (minus our sap)” —both sides count sap; give pessimistic/optimistic bounds for coarse screening. Sap has range (3–7, Chebyshev) and self-cost; focus fire is many-to-one (breaks Hungarian’s one-to-one; the first version keeps one-to-one and does focus fire greedily).
- **Greedy sap scheduling (bypassing value):** target tile collision short $\rightarrow$ in-range units focus-fire the defender; held tile under attack $\rightarrow$ sap the attacker nearby; surplus units $\rightarrow$ around the tile, sap in-range threats. Constraints: a sapping unit keeps enough energy to reach its target; surplus units keep half. $\text{value}(g)$ touches RL modeling; the rule version uses greedy thresholds to avoid it.
- **Own-path conflicts:** many tiles $\rightarrow$ dispersed, rare collisions; few tiles $\rightarrow$ greedy nearest; still blocked $\rightarrow$ “nearest-to-target moves first” to break it (not chasing MAPF/CTAPF optimality—also hard).
- **Later chapters:** rule-based, the runner-up’s lean RL (ResNet + self-play + win-only reward), imitation learning (U-Net on top replays), structured RL (belief + information gain + equivariance). Experiments mainly via local head-to-head.

All mechanics verified against the official source; the formalization, layering, complexity placement, and single-cell equilibrium are the author’s derivation. An open hand already hits a wall of PSPACE/NEXP/NP barriers—exactly why top solutions go approximate. Corrections welcome.